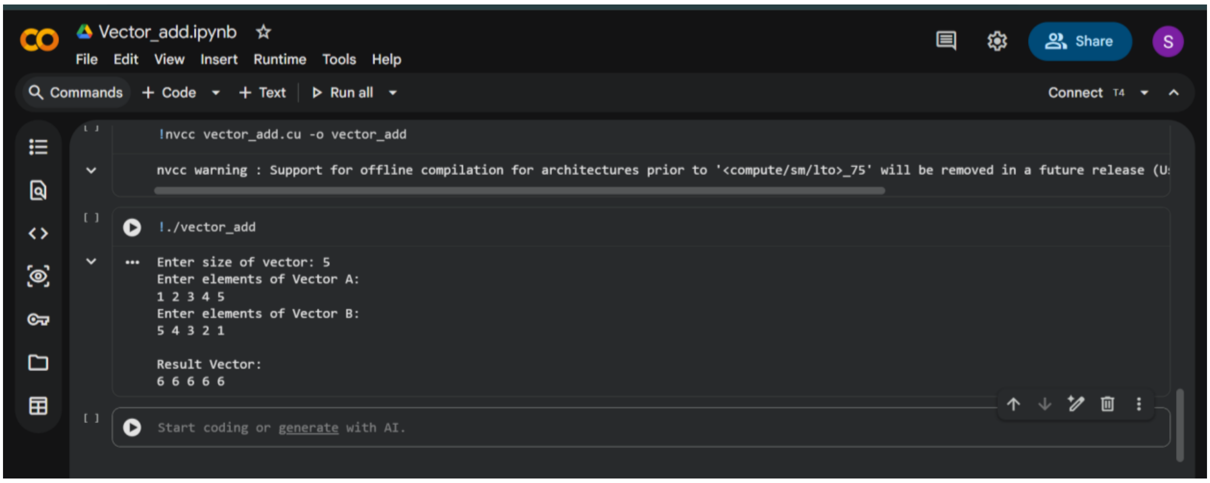
PROBLEM STATEMENT: VECTOR ADDITION

```
%%writefile vector_add.cu
#include <iostream>
#include <cuda_runtime.h>
using namespace std;

__global__ void vectorAdd(int *A, int *B, int *C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    int n;

    cout << "Enter size of vector: ";
    cin >> n;

    int size = n * sizeof(int);

    int *h_A = new int[n];
    int *h_B = new int[n];
    int *h_C = new int[n];

    cout << "Enter elements of Vector A:\n";
    for(int i = 0; i < n; i++) {
```

```
    cin >> h_A[i];
}

cout << "Enter elements of Vector B:\n";
for(int i = 0; i < n; i++) {
    cin >> h_B[i];
}

int *d_A, *d_B, *d_C;

cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, n);

cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

cout << "\nResult Vector:\n";
for(int i = 0; i < n; i++) {
    cout << h_C[i] << " ";
}
```

```
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    delete[] h_A;
    delete[] h_B;
    delete[] h_C;

    return 0;
}
```

OUTPUT:



%%writefile parallel_gpu.cu

 # Compile the CUDA code

!nvcc parallel_gpu.cu -o parallel_gpu

 # Run the program

!./parallel_gpu